



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Παράλληλα Συστήματα
Εργασία 1(2025-2026)

Ονοματεπώνυμο : Ζουμπουρλής Θοδωρής

Αριθμός Μητρώου : 1115202200267

Ονοματεπώνυμο : Τσάκωνας Βασίλης

Αριθμός Μητρώου : 1115202200245

Περιεχόμενα

1	Πληροφορίες για υπολογιστικό σύστημα	2
2	Άσκηση 1.1	2
3	Άσκηση 1.2	5
4	Άσκηση 1.3	9
5	Άσκηση 1.4	11
6	Άσκηση 1.5	21

1 Πληροφορίες για υπολογιστικό σύστημα

Όνομα υπολογιστικού συστήματος: linux02.di.uoa.gr

Μοντέλο επεξεργαστή: Intel(R) Core(TM) i7-6700 CPU @ 3.40GHz

Ο επεξεργαστής περιέχει 4 φυσικούς πυρήνες, και κάθε πυρήνας υποστηρίζει 2 νήματα μέσω Hyper-Threading.

Επομένως, διαθέτει 8 λογικούς επεξεργαστές που μπορούν να εκτελούν 8 νήματα ταυτόχρονα.

Έκδοση μεταγλωττιστή: gcc 9.4.0 (Ubuntu 9.4.0-1ubuntu1~20.04.2)

Αφού το linux02 υποστηρίζει μέχρι 8 νήματα να τρέχουν ταυτόχρονα εμείς τεστάρουμε για 1, 2, 4, 8 και 16 threads.

Επισημάνσεις Στους επιμέρους φακέλους μπορεί κάποιος να βρει τα scripts που χρησιμοποιήθηκαν για τη συλλογή, παραγωγή και ανάλυση των αποτελεσμάτων. Το κάθε script το τρέξαμε 5 φορές και έπειτα συλλέξαμε τα αποτελέσματα και παρουσιάζουμε τον μέσο όρο τους. Επιπλέον, υπάρχουν και τα python scripts που χρησιμοποιήθηκαν για τη παραγωγή των graphs.

2 Άσκηση 1.1

Πρόβλημα και Υλοποίηση

Το πρόβλημα που καλούμαστε να υλοποιήσουμε είναι ο πολλαπλασιασμός πολυωνύμων. Αυτό που κάνουμε είναι ότι στην αρχή υπολογίζουμε το αποτέλεσμα με έναν σειριακό αλγόριθμο και στη συνέχεια ανάλογα με τον αριθμό των νημάτων που έχουμε δώσουμε ως είσοδο υπολογίζουμε πάλι το ίδιο πολυώνυμο με έναν παράλληλο αλγόριθμο. Σε αυτόν το κάθε νήμα αναλαμβάνει να πολλαπλασιάσει κάποια στοιχεία της εκάστοτε πράξης. Δεν χρειάζεται κάποιος συγχρονισμός καθώς τα νήματα δε πρόκειται να πάνε να διαβάσουν ή να γράψουν στο ίδιο στοιχείο οπότε δε συμβαίνει κάποιο overall θέσεων. Επομένως, δεν έχει γίνει χρήση mutex ή rwlock αφού δεν υπάρχει κάποια εξάρτηση μεταξύ των δεδομένων.

Τέλος συγκρίνουμε το αποτέλεσμα του σειριακού με του παράλληλου αλγόριθμου για να επιβεβαιώσουμε πως είναι το ίδιο. Αν αυτό ισχύει το πρόγραμμα τότε είναι ορθό.

Polynomial Degree	10	10	10	10	10
Number of Threads	1	2	4	8	16
Initialization Time	0.000004	0.000004	0.000004	0.000004	0.000004
Serial Time	0.000002	0.000002	0.000002	0.000002	0.000002
Parallel Time	-	0.000742	0.000748	0.001027	0.001564
Speedup	1.00	0.002695	0.002674	0.001947	0.001279
Efficiency	1.00	0.001348	0.000668	0.000243	0.000080

Polynomial Degree	100	100	100	100	100
Number of Threads	1	2	4	8	16
Initialization Time	0.000016	0.000016	0.000016	0.000016	0.000017
Serial Time	0.000113	0.000112	0.000113	0.000112	0.000114
Parallel Time	-	0.000972	0.000834	0.001096	0.001529
Speedup	1.00	0.12	0.14	0.10	0.07
Efficiency	1.00	0.06	0.03	0.01	0.00

Polynomial Degree	1000	1000	1000	1000	1000
Number of Threads	1	2	4	8	16
Initialization Time	0.000173	0.000176	0.000177	0.000173	0.000171
Serial Time	0.011031	0.011832	0.010805	0.011426	0.010852
Parallel Time	-	0.012981	0.012191	0.007668	0.006061
Speedup	1.00	0.91	0.89	1.49	1.79
Efficiency	1.00	0.46	0.22	0.19	0.11

Polynomial Degree	10000	10000	10000	10000	10000
Number of Threads	1	2	4	8	16
Initialization Time	0.001163	0.000452	0.000519	0.000707	0.000417
Serial Time	0.262597	0.248942	0.249768	0.249169	0.248997
Parallel Time	-	0.287881	0.227005	0.159417	0.151427
Speedup	1.00	0.86	1.10	1.56	1.64
Efficiency	1.00	0.43	0.28	0.20	0.10

Polynomial Degree	100000	100000	100000	100000	100000
Number of Threads	1	2	4	8	16
Initialization Time	0.003507	0.005611	0.003712	0.004408	0.006172
Serial Time	24.419215	24.419530	24.430913	24.431349	24.453446
Parallel Time	-	28.740097	17.160884	15.099710	14.235694
Speedup	1.00	0.85	1.42	1.62	1.72
Efficiency	1.00	0.42	0.36	0.20	0.11

Ανάλυση Αποτελεσμάτων

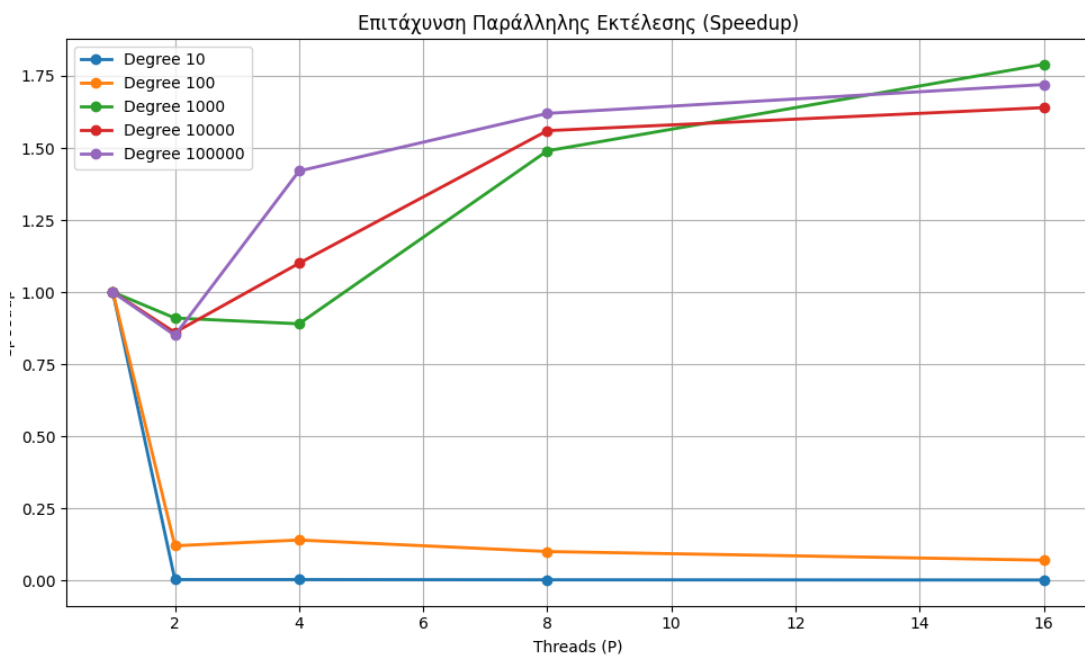
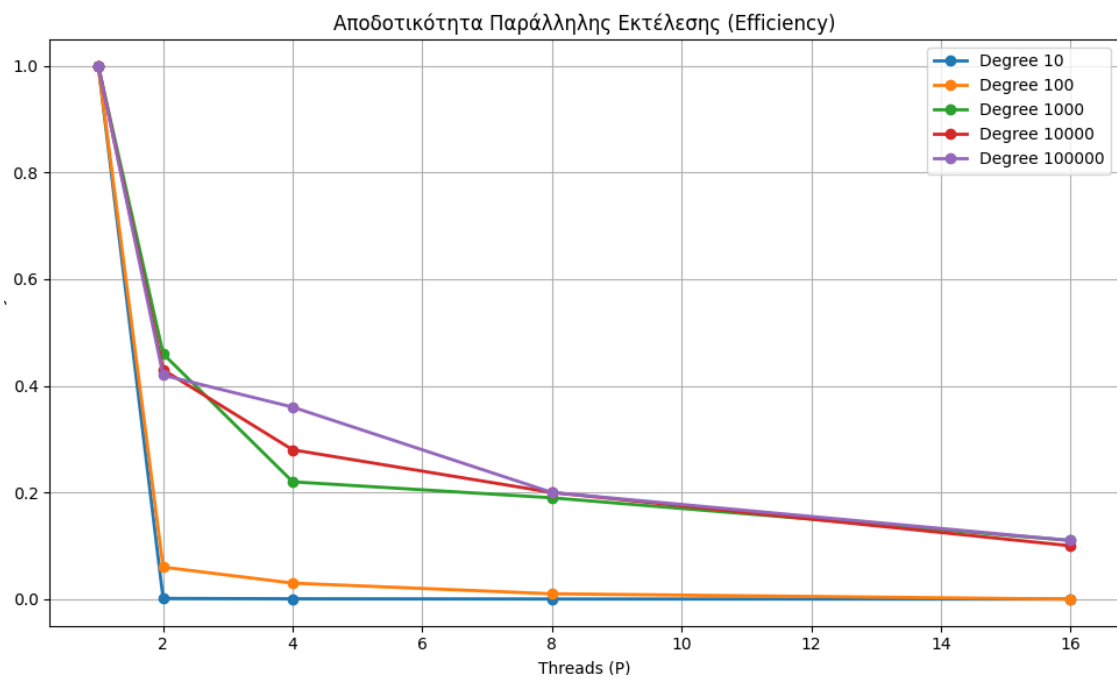
Παραπάνω παρουσιάζονται τα αποτελέσματα για Threads 1, 2, 4, 8, 16, και με βαθμό πολυωνύμων 10, 100, 1000, 10000, 100000.

Αρχικά παρατηρούμε ότι για πολύ μικρά προβλήματα 10 και 100 βαθμό πολυωνύμων, το κόστος της δημιουργίας και συγχρονισμού νημάτων υπερβαίνει το όφελος $\text{Speedup} = 0$ και $\text{Efficiency} = 0$. Οι παράλληλοι χρόνοι είναι μεγαλύτεροι από τους σειριακούς, το οποίο είναι το αναμενόμενο καθώς η "δουλειά" που κάνει το κάθε νήμα είναι μικρή οπότε μόνο και μόνο ο χρόνος δημιουργίας των threads, το scheduling και το synchronization overhead είναι πολύ μεγαλύτερα από την πραγματική "δουλειά" που εκτελεί το κάθε thread. Επομένως η παραλληλοποίηση δεν είναι χρήσιμη για μικρά workloads και φαίνεται από το γεγονός ότι $\text{speedup} < 1$ και το efficiency είναι πολύ μικρό.

Για μεγέθη της τάξεως των 1000 και 10000 η διαφορά σειριακού-παράλληλου χρόνου είναι σημαντική. Για μεγαλύτερου βαθμού πολυώνυμα παρατηρούμε πως όσο μεγαλύτερος είναι ο βαθμός τόσο περισσότερο αποδίδει να χρησιμοποιήσουμε περισσότερα νήματα αφού η δουλειά σπάει στα επιμέρους νήματα και συμφέρει αφού ο φόρτος είναι αρκετά μεγάλος.

Τέλος για 100000 παρατηρούμε ότι οι Ύψηλοί σειριακοί χρόνοι $\rightarrow$ μεγάλο φορτίο υπολογισμών, αναδεικνύουν στο μέγιστο την διαφορά ανάμεσα στο Serial και το Parallel time. Οι χρόνοι του speedup και Efficiency είναι οι αναμενόμενοι λόγω overhead και νόμου του Amdahl. Επομένως, οι μεγαλύτεροι βαθμοί πολυωνύμων αξιοποιούν πολύ καλύτερα την παραλληλοποίηση.

Παρακάτω σας παραθέτουμε 2 γράφους για το speedup και το efficiency.



3 Άσκηση 1.2

Πρόβλημα και Υλοποίηση

Σε αυτή την άσκηση, το πρόβλημα που καλούμαστε να υλοποιήσουμε είναι η αύξηση της τιμής μιας κοινόχρηστης μεταβλητής n φορές από πολλά νήματα ταυτόχρονα. Ο αριθμός των νημάτων δίνεται ως είσοδος από τον χρήστη. Το πρόβλημα που δημιουργείται στην προκειμένη περίπτωση είναι τα race conditions μεταξύ των νημάτων αφού μπορεί να μην ανανεωθεί σωστά η τιμή αν δε τα συγχρονίσουμε. Επομένως, τεστάρουμε 3 διαφορετικούς μηχανισμούς συγχρονισμού: με τη χρήση κλειδωμάτων αμοιβαίου αποκλεισμού (MUTEX), με τη χρήση κλειδωμάτων ανάγνωσης-εγγραφής (RWLOCK), και με τη χρήση ατομικών εντολών (atomic operations). Επιπλέον, κάνουμε κάποιες δοκιμές με τον αριθμό των επαναλήψεων για να παρατηρήσουμε ποιος συγχρονισμός είναι ο πιο γρήγορος και αποδοτικός.

Παρακάτω είναι μαζεμένα στα πινακάκια οι χρόνοι και οι αποδόσεις ανάλογα με την υλοποίηση που έχουμε κάνει και οι παρατηρήσεις μας.

i) Υλοποίηση με τη χρήση κλειδωμάτων αμοιβαίου αποκλεισμού

Threads	Loops per thread									
	100		1000		10000		100000		1000000	
	Time	Eff.	Time	Eff.	Time	Eff.	Time	Eff.	Time	Eff.
1	0.00027	1.00	0.00035	1.00	0.00092	1.00	0.0067	1.00	0.046	1.00
2	0.00025	0.54	0.00031	0.56	0.00270	0.17	0.030	0.11	0.056	0.41
4	0.00032	0.21	0.00120	0.073	0.01200	0.019	0.035	0.048	0.16	0.072
8	0.00064	0.053	0.00260	0.034	0.02500	0.011	0.087	0.0096	0.61	0.0094
16	0.00120	0.014	0.00530	0.0066	0.02500	0.0023	0.18	0.0023	1.36	0.002114

Table 1: Χρόνος σε δευτερόλεπτα και απόδοση για την υλοποίηση με mutex

ii) Υλοποίηση με τη χρήση κλειδωμάτων ανάγνωσης-εγγραφής

Threads	Loops per thread									
	100		1000		10000		100000		1000000	
	Time	Eff.	Time	Eff.	Time	Eff.	Time	Eff.	Time	Eff.
1	0.00029	1.00	0.00039	1.00	0.00044	1.00	0.011	1.00	0.027	1.00
2	0.00025	0.58	0.00082	0.238	0.0071	0.031	0.067	0.082	0.15	0.09
4	0.00043	0.17	0.0014	0.069	0.015	0.0073	0.108	0.025	0.383	0.018
8	0.00067	0.054	0.0041	0.012	0.0315	0.00175	0.167	0.0082	1.123	0.003
16	0.0012	0.015	0.034	0.00072	1.016	0.000027	12.15	0.000057	65.20	0.000026

Table 2: Χρόνος σε δευτερόλεπτα και απόδοση για την υλοποίηση με RWLOCK .

iii) Υλοποίηση με τη χρήση ατομικών εντολών

Threads	Loops per thread									
	100		1000		10000		100000		1000000	
	Time	Eff.	Time	Eff.	Time	Eff.	Time	Eff.	Time	Eff.
1	0.00029	1.000	0.00016	1.000	0.00039	1.000	0.0017	1.000	0.015	1.000
2	0.00026	0.558	0.00015	0.533	0.0016	0.122	0.012	0.071	0.11	0.068
4	0.00044	0.165	0.00038	0.105	0.0028	0.035	0.019	0.022	0.12	0.031
8	0.00064	0.057	0.00068	0.029	0.0063	0.008	0.053	0.004	0.16	0.012
16	0.0012	0.015	0.0011	0.009	0.013	0.002	0.09	0.001	0.32	0.003

Table 3: Time και Efficiency για το atomic operations implementation.

Συγκρίνοντας τις τρεις υλοποιήσεις συμπεραίνουμε πως αυτή με τη χρήση ατομικών εντολών είναι η πιο γρήγορη και αποδοτική, ειδικά για μεγάλο αριθμό επαναλήψεων και μεγάλο πλήθος νημάτων που τρέχουν παράλληλα.

Αυτό συμβαίνει διότι χρησιμοποιεί απευθείας CPU atomic instructions, όπου ο συγχρονισμός γίνεται σε επίπεδο hardware χωρίς εμπλοκή του λειτουργικού συστήματος. Έτσι, τα νήματα δεν μπλοκάρουν και κλειδώνεται μόνο η αντίστοιχη cache line της μεταβλητής, γεγονός που εξηγεί τους σημαντικά καλύτερους χρόνους ακόμα και για 8 ή 16 νήματα.

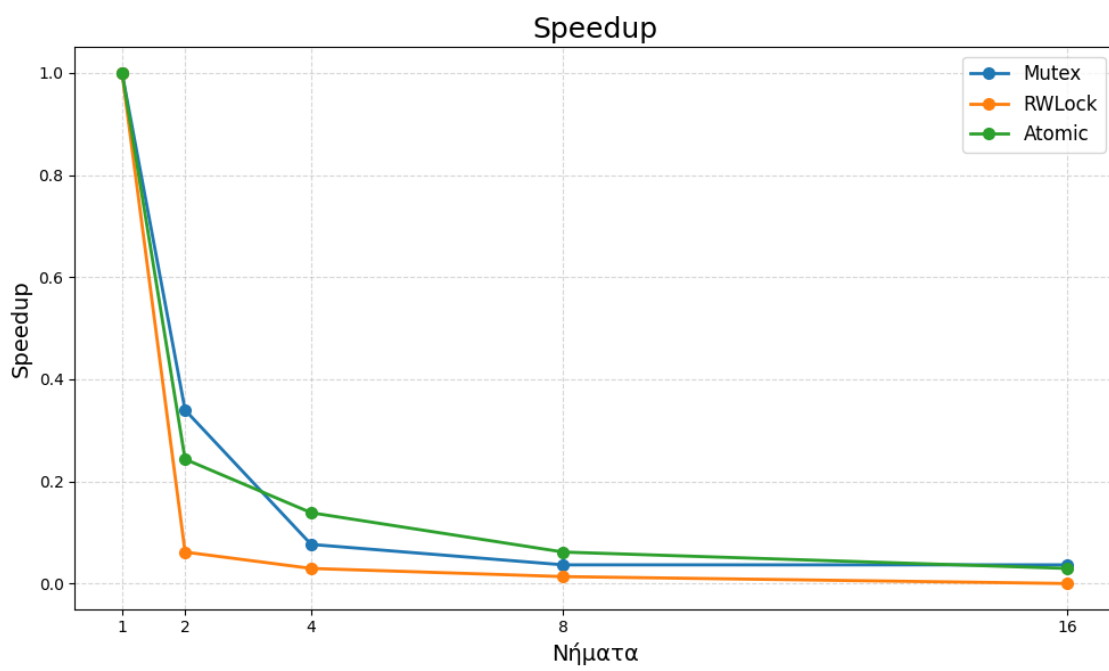
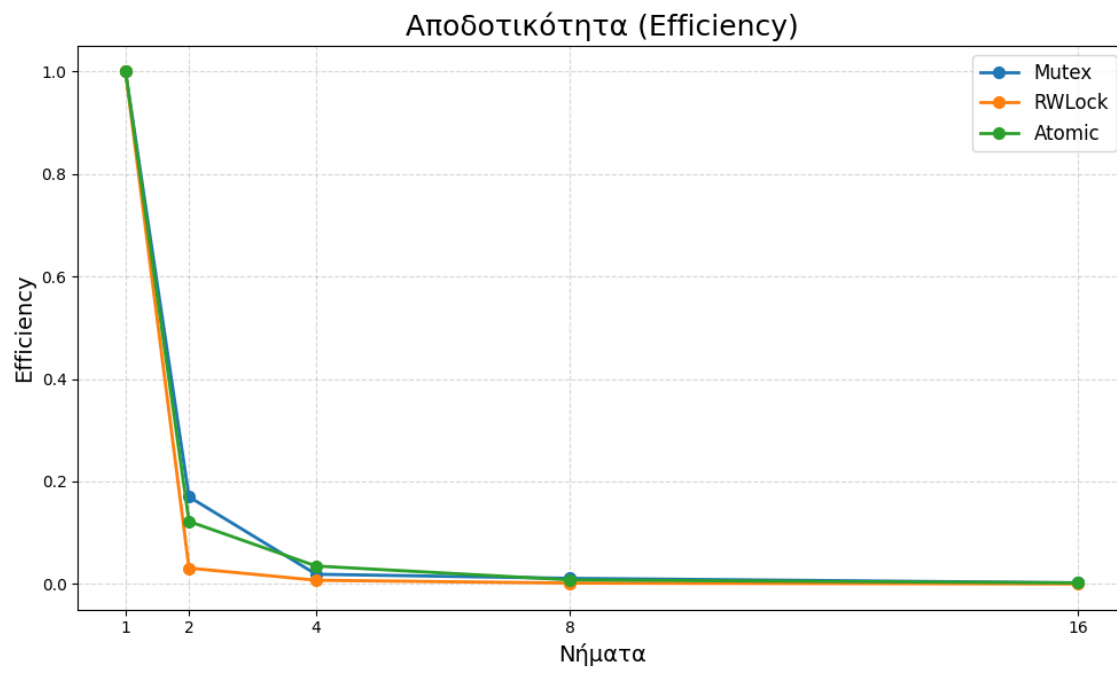
Η δεύτερη καλύτερη υλοποίηση είναι αυτή με χρήση κλειδωμάτων αμοιβαίου αποκλεισμού (mutex). Ενώ προσφέρει ορθότητα, παρουσιάζει σημαντική επιβάρυνση λόγω του ότι κάθε νήμα που εισέρχεται στο critical section μπλοκάρει πλήρως τα υπόλοιπα. Το κόστος του lock/unlock επηρεάζει έντονα τις επιδόσεις όσο αυξάνονται τα threads και τα loops.

Η χειρότερη υλοποίηση — με σημαντικά μεγαλύτερους χρόνους για μεγάλο αριθμό νημάτων και επαναλήψεων — είναι αυτή με κλειδώματα ανάγνωσης-εγγραφής (RWLock). Στο συγκεκριμένο πρόγραμμα όλα τα locks είναι write locks, οπότε το RWLock συμπεριφέρεται πρακτικά σαν mutex, αλλά με μεγαλύτερο overhead. Ο μηχανισμός αυτός περιλαμβάνει πρόσθετη διαχείριση δομών για readers/writers και γενικά αποδίδει καλά μόνο σε σενάρια με πολλούς αναγνώστες και λίγους writers, κάτι που δεν ισχύει εδώ.

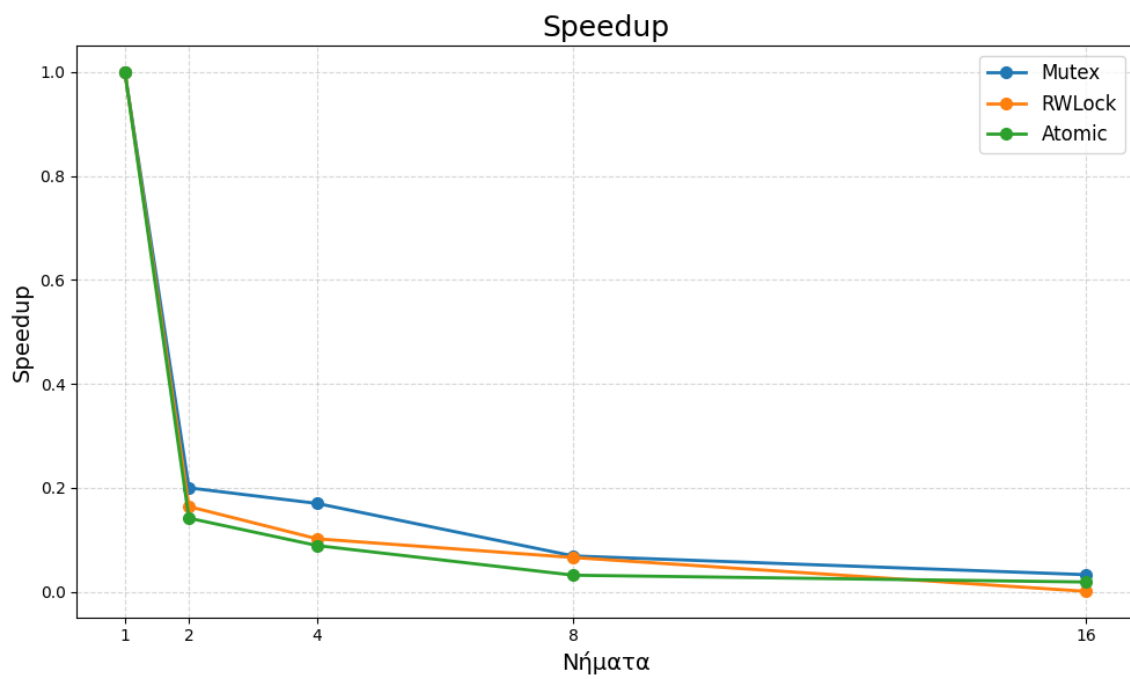
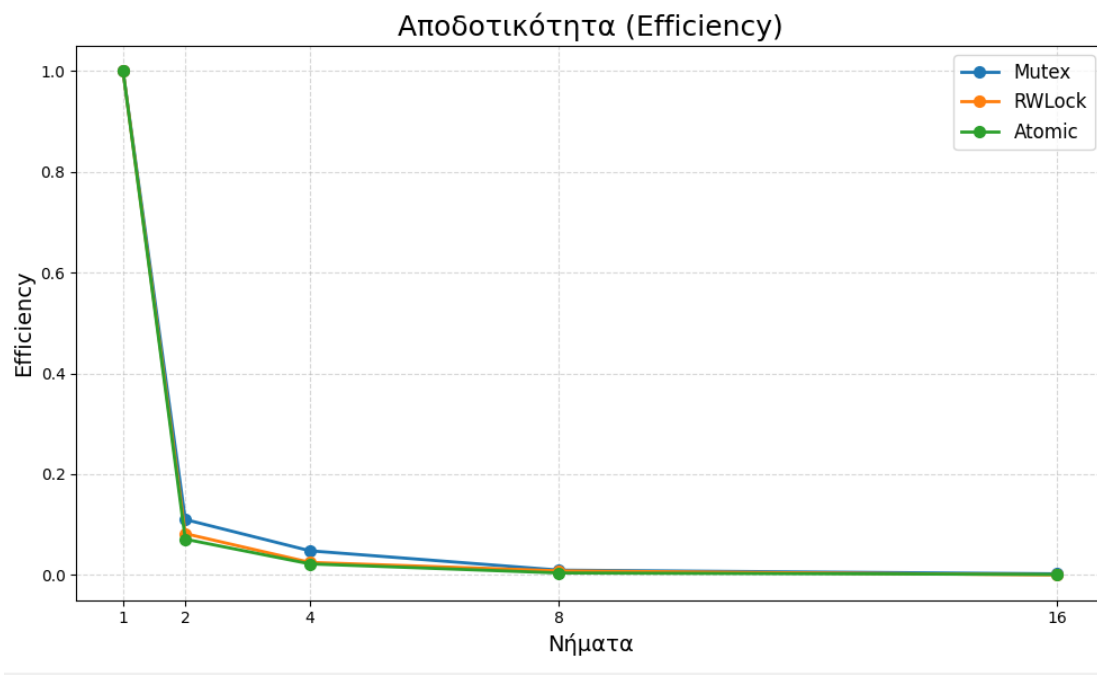
Τέλος, παρατηρούμε πως όσα περισσότερα νήματα χρησιμοποιούμε τόσο μεγαλύτερος είναι ο χρόνος και αυτό είναι το αναμενόμενο καθώς στη πραγματικότητα δε γίνεται κάποια δουλεία παράλληλα. Όλα τα νήματα πάνε και αλλάζουν την ίδια μεταβλητή πολλές φορές. Έτσι, η δουλεία είναι σαν να γίνεται σειριακά προσθέτοντας επιπλέον και τον αντίστοιχο χρόνο που απαιτείται για το κλείδωμα και το ξεκλείδωμα στο critical section.

Παρακάτω παραθέτουμε και τα γραφήματα αποδοτικότητας (Efficiency) και speedup για 10000 και 100000 επαναλήψεις αντίστοιχα.

Για 10000 επαναλήψεις:



Για 100000 επαναλήψεις:

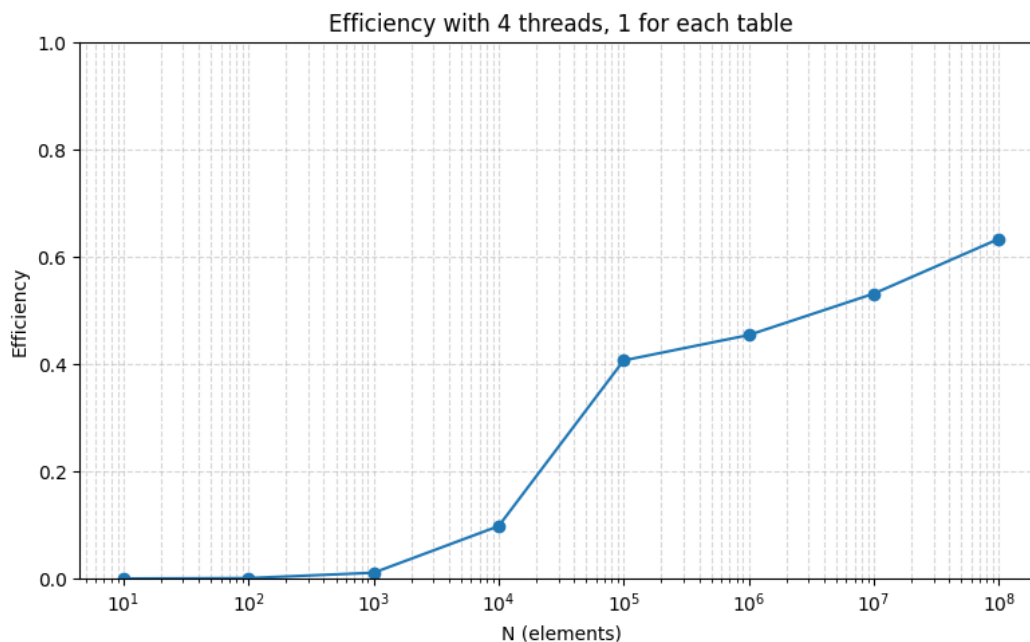


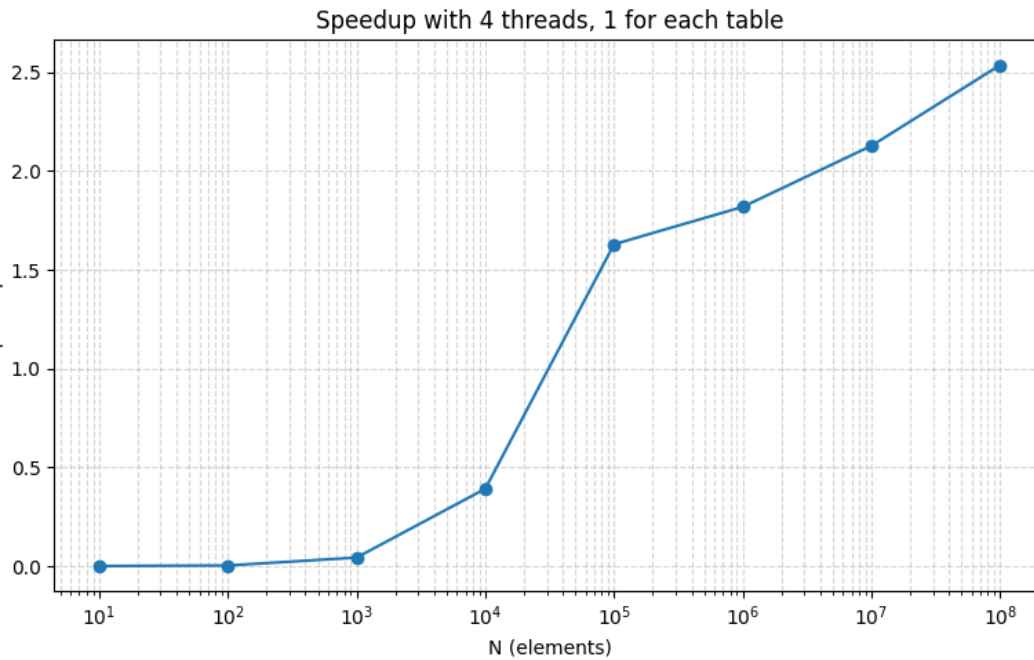
4 Άσκηση 1.3

Πρόβλημα και Υλοποίηση

Στην άσκηση καλούμαστε να μετρήσουμε τα μη-μηδενικά στοιχεία 4 πινάκων με σειριακό τρόπο (ένα μόνο νήμα επεξεργάζεται διαδοχικά όλους τους πίνακες και μετρά τα μη-μηδενικά στοιχεία.) και με παράλληλο τρόπο. Στον παράλληλο τρόπο φτιάχνουμε 4 νήματα όπου το κάθε νήμα αναλαμβάνει από έναν πίνακα. Για την αποθήκευση του αριθμού των μη-μηδενικών στοιχείων χρησιμοποιούμε το struct `array_stats_s` που περιέχει τη μεταβλητή `info_array_n` για $n = 0,1,2,3$. Δεν απαιτείται κάποιος μηχανισμός συγχρονισμού καθώς στην προκειμένη περίπτωση δε δημιουργείται κάποιο critical section αφού το κάθε νήμα επεξεργάζεται διαφορετικό πίνακα του και στο τέλος ενημερώνει μια μεταβλητή που αντιστοιχεί στον εκάστοτε πίνακα, οπότε το κάθε νήμα έχει τη δική του μεταβλητή. Στο τέλος, συγκρίνουμε τα επιμέρους αποτελέσματα των 2 τρόπων (παράλληλος και σειριακός) για να βεβαιωνούμε πως η διαδικασία είναι ορθή. Παρακάτω παραθέτουμε τα αποτελέσματα και τις παρατηρήσεις μου.

N (elements)	Initialization Time (sec)	Serial Time (sec)	Parallel Time (sec)	Speedup	Efficiency
10	0.000009	0.000001	0.000826	0.000828	0.00
100	0.00002	0.000003	0.000715	0.00449	0.00
1000	0.000173	0.000035	0.000793	0.0439	0.01
10000	0.001871	0.000337	0.000859	0.392	0.10
100000	0.016190	0.00209	0.00128	1.629	0.41
1000000	0.055501	0.00802	0.00440	1.819	0.45
10000000	0.412910	0.08001	0.03761	2.127	0.53
100000000	4.007875	0.7794	0.3074	2.536	0.63





Ανάλυση Αποτελεσμάτων

Την υλοποίηση μας την δοκιμάσαμε σε έναν πίνακα που περιέχει 10, 100, 1000, 10000, 10^5 , 10^6 , 10^7 και 10^8 στοιχεία. Αυτό που συμπεραίνουμε είναι πως για μικρά μεγέθη πινάκων (έως 10000 στοιχεία) το parallel time είναι μεγαλύτερο του serial. Το speedup και η απόδοση βελτιώνεται όσο περισσότερα είναι τα στοιχεία του πίνακα ($\geq 10^5$). Αυτό είναι και το αναμενόμενο αφού για μικρό n στον παράλληλο αλγόριθμο το workload του εκάστοτε νήματος είναι πολύ μικρό. Όσο μεγαλύτερος είναι ο αριθμός των στοιχείων το workload των νημάτων τόσο αυξάνεται και το speedup με την απόδοση, αφού ο χρόνος επεξεργασίας γίνεται πλέον σημαντικός συγκριτικά με το σταθερό overhead των threads.

Ένα πρόβλημα που μπορεί να προκύπτει στη δομή `array_stats` είναι το false sharing. Ουσιαστικά τα 4 πεδία βρίσκονται συνεχόμενα στην μνήμη με αποτέλεσμα να είναι πιθανό να είναι στην ίδια cache line και αυτή να πηγαينόερχεται συνέχεια μεταξύ των πυρήνων του επεξεργαστή. Αυτό μπορεί να λυθεί με τη χρήση padding. Έτσι η κάθε μεταβλητή θα έχει σίγουρα το δικό της cache line και η επίδοση της παραλληλίας θα είναι καλύτερη σε σχέση με πριν.

5 Άσκηση 1.4

Πρόβλημα και Υλοποίηση

Σε αυτή την άσκηση καλούμαστε να φτιάξουμε ένα σύστημα που είναι παρόμοιο με αυτό μιας τράπεζας. Υπάρχει ένας κοινόχρηστος πίνακας όπου το κάθε στοιχείο του περιέχει το χρηματικό υπόλοιπο ενός λογαριασμού. Ανάλογα με την είσοδο του προγράμματος θα πραγματοποιούνται συναλλαγές και ερωτήσεις υπολοίπων για κάποιους τυχαίους λογαριασμούς. Επειδή 2 νήματα μπορεί να θέλουν να έχουν πρόσβαση στο ίδιο στοιχείο του πίνακα την ίδια στιγμή θα πρέπει να συγχρονίσουμε το πρόβλημα μας. Επομένως, δοκιμάζουμε 4 τρόπους συγχρονισμού:

- coarse-grained με χρήση κλειδωμάτων αμοιβαίου αποκλεισμού (Σε κάθε συναλλαγή και ερώτηση υπολοίπου κλειδώνει ολόκληρος ο πίνακας με έναν global mutex)
- coarse-grained με χρήση κλειδωμάτων ανάγνωσης-εγγραφής (Σε κάθε συναλλαγή κλειδώνει όλος ο πίνακας με wrlock και ερώτηση υπολοίπου κλειδώνει ολόκληρος ο πίνακας με έναν rdlock)
- fine-grained με χρήση κλειδωμάτων αμοιβαίου αποκλεισμού (Σε κάθε συναλλαγή κλειδώνουν μόνο οι θέσεις που παίρνουν μέρος στην συναλλαγή και σε ερωτήσεις υπολοίπου κλειδώνει μόνο η θέση πάνω στην οποία γίνεται η ερώτηση. Η κάθε θέση έχει τον δικό της Mutex)
- fine-grained με χρήση κλειδωμάτων ανάγνωσης-εγγραφής (Σε κάθε συναλλαγή κλειδώνουν μόνο οι θέσεις που παίρνουν μέρος στην συναλλαγή με wrlock και σε ερωτήσεις υπολοίπου κλειδώνει με rdlock μόνο η θέση πάνω στην οποία γίνεται η ερώτηση. Η κάθε θέση έχει τον δικό της rwlock)

Παρακάτω παραθέτουμε τα αποτελέσματα και τις παρατηρήσεις μου.

Lock Type	Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained (mutex)	1	0.001948	1.00	1.000
	2	0.013666	0.14	0.070
	4	0.029639	0.07	0.017
	8	0.033003	0.06	0.007
	16	0.042901	0.05	0.003
coarse-grained (RW-lock)	1	0.0005910	1.00	1.000
	2	0.0037711	0.16	0.080
	4	0.0141501	0.04	0.010
	8	0.0245130	0.02	0.003
	16	0.2599201	0.002	0.0001
fine-grained (mutex)	1	0.001092	1.00	1.000
	2	0.001100	0.99	0.495
	4	0.001721	0.63	0.157
	8	0.003058	0.36	0.045
	16	0.009306	0.12	0.0075
fine-grained (RW-lock)	1	0.005830	1.00	1.000
	2	0.006513	0.89	0.445
	4	0.009097	0.64	0.160
	8	0.010741	0.54	0.067
	16	0.017851	0.33	0.020

Table 4: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 100000, συναλλαγές = 10000, ποσοστό για ερωτήσεις υπολοίπου = 10%

Lock Type	Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained (mutex)	1	0.0013856	1.00	1.000
	2	0.0033522	0.41	0.207
	4	0.0163660	0.08	0.021
	8	0.0311770	0.04	0.006
	16	0.0595510	0.02	0.001
coarse-grained (RW-lock)	1	0.0010550	1.00	1.000
	2	0.005292	0.20	0.100
	4	0.0251720	0.04	0.010
	8	0.031745	0.03	0.004
	16	0.3944411	0.00	0.000
fine-grained (mutex)	1	0.001849	1.00	1.000
	2	0.001942	0.95	0.476
	4	0.003233	0.57	0.143
	8	0.005323	0.35	0.043
	16	0.007364	0.25	0.016
fine-grained (RW-lock)	1	0.004374	1.00	1.000
	2	0.004474	0.98	0.489
	4	0.005849	0.75	0.187
	8	0.006651	0.66	0.082
	16	0.011353	0.39	0.024

Table 5: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 1000000, συναλλαγές =10000, ποσοστό για ερωτήσεις υπολοίπου =10%

Lock Type	# Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained (mutex)	1	0.00130	1.00	1.00
	2	0.0140	0.0928	0.0464
	4	0.0186	0.0700	0.0175
	8	0.0425	0.0306	0.00382
	16	0.0855	0.0152	0.000950
coarse-grained (RW-lock)	1	0.00134	1.00	1.00
	2	0.00686	0.196	0.0981
	4	0.0203	0.0663	0.0166
	8	0.0374	0.0360	0.00450
	16	1.86	0.000722	0.0000451
fine-grained (mutex)	1	0.00235	1.00	1.00
	2	0.00232	1.01	0.506
	4	0.00357	0.658	0.164
	8	0.00610	0.385	0.0481
	16	0.0105	0.224	0.0140
fine-grained (RW-lock)	1	0.0217	1.00	1.00
	2	0.0215	1.01	0.504
	4	0.0233	0.932	0.233
	8	0.0269	0.807	0.101
	16	0.0300	0.723	0.0452

Table 6: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 10000000, συναλλαγές = 10000, ποσοστό για ερωτήσεις υπολοίπου =10%

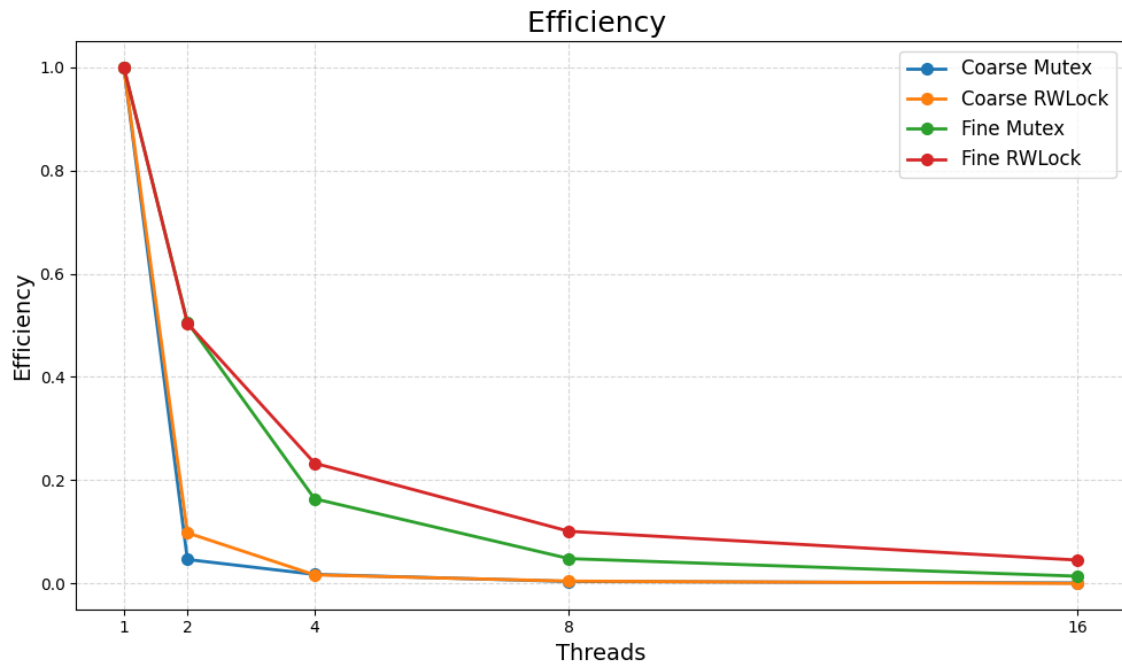


Figure 1: Efficiency για μέγεθος πίνακα = 10000000, συναλλαγές = 10000 και ποσοστό ερωτήσεων υπολοίπου = 10%

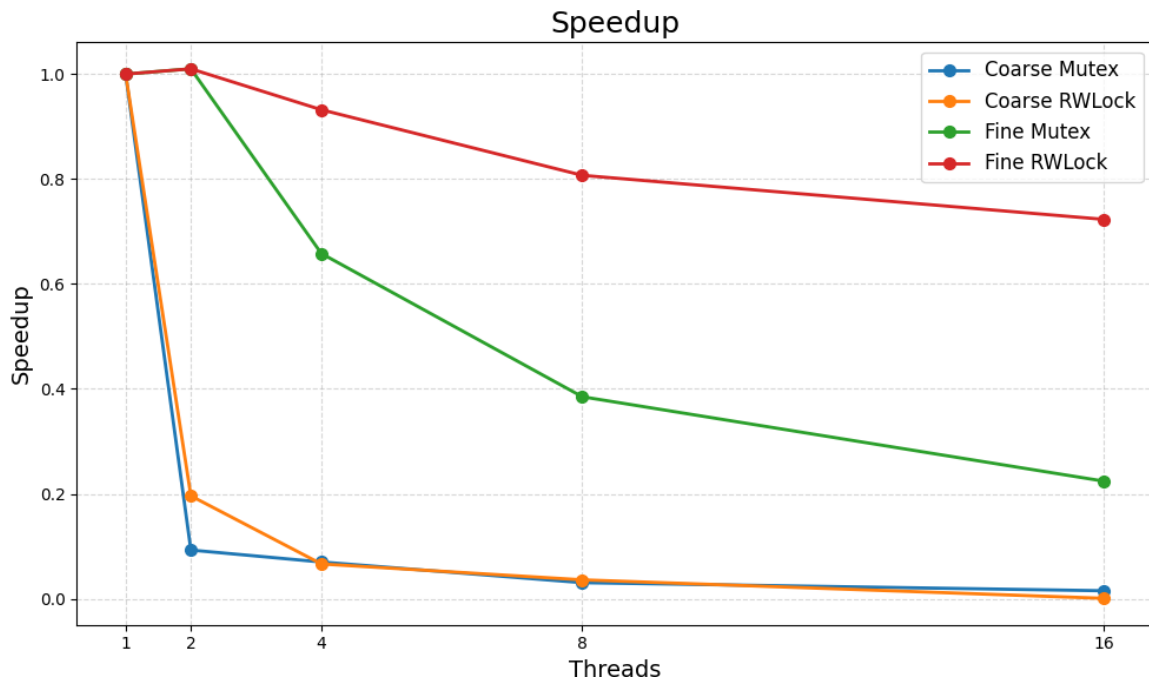


Figure 2: Speedup για μέγεθος πίνακα = 10000000, συναλλαγές = 10000 και ποσοστό ερωτήσεων υπολοίπου = 10%

Συγκρίνοντας τα παραπάνω δεδομένα κάποιος μπορεί εύκολα να συμπεράνει πως για οποιοδήποτε μέγεθος πίνακα η fine-grain υλοποίηση είναι καλύτερη. Αυτό συμβαίνει καθώς ο εκάστοτε μηχανισμός συγχρονισμού μπλοκάρει μόνο τα νήματα που πάνε να αποκτήσουν πρόσβαση στην ίδια θέση πίνακα ενώ στην coarse-grain ουσιαστικά για κάθε συναλλαγή ή ερώτηση μπλοκάρουν όλα τα νήματα αφού μόνο ένα νήμα μπορεί να έχει πρόσβαση στον κοινόχρηστο πίνακα. Επιπλέον για σταθερό ποσοστό ερωτήσεις υπολοίπου και σταθερό αριθμό συναλλαγών βλέπουμε μόνο μικροδιαφορές μεταξύ του τύπου κλειδώματος (mutex και rwlock), επειδή το read critical section είναι πολύ σύντομο (παραπάνω ανάλυση πιο κάτω). Συνεχίζουμε τώρα διαφοροποιώντας τον αριθμό των συναλλαγών κρατώντας τα υπόλοιπα μεγέθη σταθερά.

Lock Type	Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained (mutex)	1	0.00132	1.00	1.00
	2	0.0127	0.104	0.052
	4	0.0201	0.0654	0.0164
	8	0.0430	0.0307	0.00384
	16	0.0844	0.0156	0.000975
coarse-grained (RW-lock)	1	0.00143	1.00	1.00
	2	0.00720	0.198	0.0991
	4	0.0257	0.0556	0.0139
	8	0.0378	0.0378	0.00472
	16	2.34	0.000610	0.0000381
fine-grained (mutex)	1	0.00230	1.00	1.00
	2	0.00231	0.996	0.498
	4	0.00352	0.653	0.163
	8	0.00587	0.392	0.0490
	16	0.00942	0.244	0.0153
fine-grained (RW-lock)	1	0.0217	1.00	1.00
	2	0.0218	0.995	0.497
	4	0.0235	0.923	0.231
	8	0.0283	0.767	0.0959
	16	0.0301	0.721	0.0451

Table 7: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 10000000, συναλλαγές = 10000, ποσοστό για ερωτήσεις υπολοίπου = 10%

Lock Type	Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained με mutex	1	0.0116	1.00	1.00
	2	0.147	0.0789	0.0394
	4	0.147	0.0791	0.0198
	8	0.419	0.0276	0.00345
	16	0.837	0.0138	0.000861
coarse-grained με RW-lock	1	0.0124	1.00	1.00
	2	0.101	0.123	0.0615
	4	0.175	0.0706	0.0177
	8	0.373	0.0332	0.00415
	16	0.769	0.0161	0.00101
fine-grained με mutex	1	0.0209	1.00	1.00
	2	0.0225	0.929	0.464
	4	0.0344	0.608	0.152
	8	0.0459	0.455	0.0569
	16	0.0958	0.218	0.0136
fine-grained με RW-lock	1	0.0506	1.00	1.00
	2	0.0530	0.955	0.477
	4	0.0716	0.707	0.177
	8	0.0851	0.595	0.0744
	16	0.145	0.349	0.0218

Table 8: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 10000000, συναλλαγές = 100000, ποσοστό για ερωτήσεις υπολοίπου = 10%

Lock Type	Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained με mutex	1	0.130	1.00	1.00
	2	1.47	0.0882	0.0441
	4	1.23	0.106	0.0264
	8	4.15	0.0313	0.00391
	16	8.42	0.0154	0.000964
coarse-grained με RW-lock	1	0.139	1.00	1.00
	2	1.00	0.138	0.0690
	4	1.13	0.123	0.0307
	8	3.65	0.0381	0.00476
	16	55.6	0.00250	0.000156
fine-grained με mutex	1	0.232	1.00	1.00
	2	0.257	0.904	0.452
	4	0.301	0.770	0.192
	8	0.471	0.492	0.0615
	16	0.960	0.241	0.0151
fine-grained με RW-lock	1	0.321	1.00	1.00
	2	0.335	0.958	0.479
	4	0.387	0.829	0.207
	8	0.599	0.536	0.0670
	16	1.15	0.279	0.0174

Table 9: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 10000000, συναλλαγές = 1000000, ποσοστό για ερωτήσεις υπολοίπου = 10%

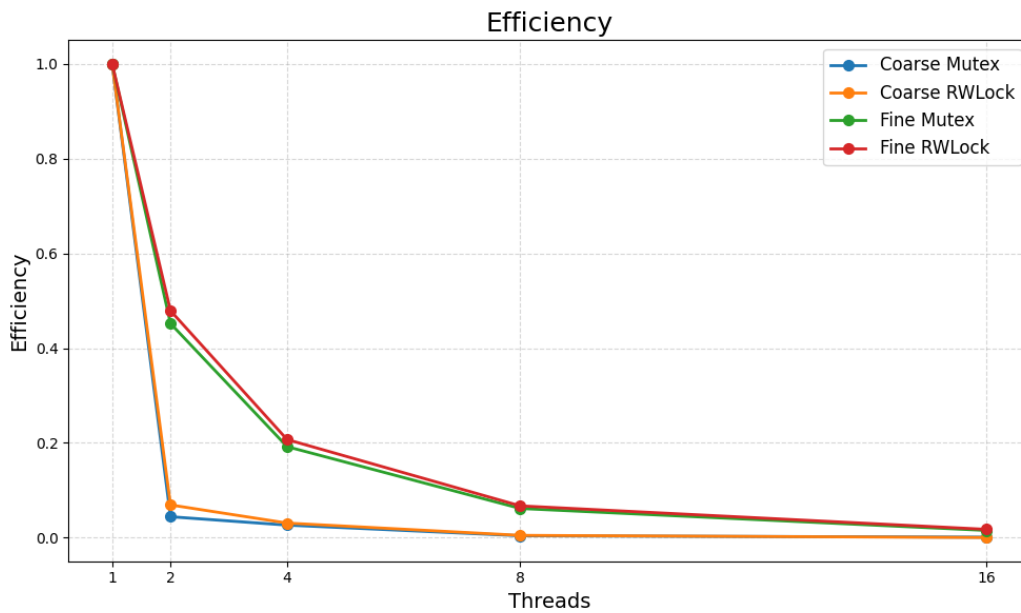


Figure 3: Efficiency για μέγεθος πίνακα = 10000000, συναλλαγές = 1000000 και ποσοστό ερωτήσεων υπολοίπου = 10%

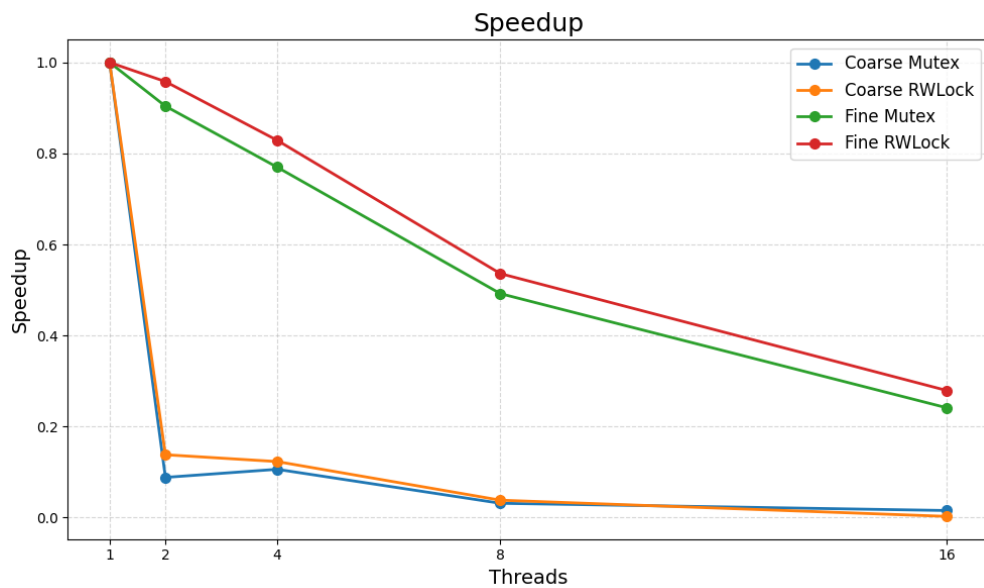


Figure 4: Speedup για μέγεθος πίνακα = 10000000, συναλλαγές = 1000000 και ποσοστό ερωτήσεων υπολοίπου = 10%

Αυξάνοντας τον συνολικό αριθμό των συναλλαγών, αυξάνονται και οι λειτουργίες που υλοποιούνται ανά νήμα. Έτσι παρατηρούμε πως η fine-grain είναι καλύτερη από τη coarse-grain υλοποίηση ανεξαρτήτως του μηχανισμού συγχρονισμού και σε αυτή τη μεταβολή (για τους ίδιους λόγους με πριν). Τώρα συνεχίζουμε μεταβάλλοντας το ποσοστό των ερωτήσεων υπολοίπου.

Lock Type	Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained με mutex	1	0.01835	1.00	1.00
	2	0.05551	0.330	0.165
	4	0.06617	0.277	0.0693
	8	0.20511	0.0894	0.0112
	16	0.40650	0.0451	0.00282
coarse-grained με RW-lock	1	0.01960	1.00	1.00
	2	0.11096	0.176	0.0878
	4	0.11225	0.174	0.0435
	8	0.40198	0.0487	0.00609
	16	2.37984	0.00824	0.000515
fine-grained με mutex	1	0.00527	1.00	1.00
	2	0.00937	0.562	0.281
	4	0.02045	0.258	0.0645
	8	0.04581	0.115	0.0144
	16	0.09875	0.0534	0.00334
fine-grained με RW-lock	1	0.01025	1.00	1.00
	2	0.02050	0.500	0.250
	4	0.02926	0.350	0.0875
	8	0.04684	0.219	0.0274
	16	0.19639	0.0522	0.00326

Table 10: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 1000, συναλλαγές = 100000, ποσοστό για ερωτήσεις υπολοίπου = 10%

Lock Type	Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained με mutex	1	0.00533	1.00	1.00
	2	0.03511	0.152	0.0761
	4	0.13467	0.0395	0.00988
	8	0.18689	0.0285	0.00356
	16	0.36779	0.0145	0.000906
coarse-grained με RW-lock	1	0.02038	1.00	1.00
	2	0.12775	0.159	0.0795
	4	0.12470	0.163	0.0408
	8	0.39616	0.0514	0.00643
	16	2.83264	0.00720	0.000450
fine-grained με mutex	1	0.01704	1.00	1.00
	2	0.02828	0.603	0.301
	4	0.04064	0.419	0.104
	8	0.05536	0.308	0.0385
	16	0.09831	0.173	0.0108
fine-grained με RW-lock	1	0.02317	1.00	1.00
	2	0.03370	0.687	0.344
	4	0.05448	0.425	0.106
	8	0.06664	0.348	0.0435
	16	0.14668	0.158	0.00987

Table 11: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 1000, συναλλαγές = 100000, ποσοστό για ερωτήσεις υπολοίπου = 40%

Lock Type	Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained με mutex	1	0.01713	1.00	1.00
	2	0.06884	0.249	0.124
	4	0.11917	0.144	0.0360
	8	0.16547	0.103	0.0129
	16	0.38569	0.0444	0.00277
coarse-grained με RW-lock	1	0.01925	1.00	1.00
	2	0.12284	0.157	0.0786
	4	0.15648	0.123	0.0308
	8	0.42564	0.0452	0.00564
	16	0.86002	0.0224	0.00140
fine-grained με mutex	1	0.01746	1.00	1.00
	2	0.02567	0.680	0.340
	4	0.03801	0.459	0.115
	8	0.05513	0.317	0.0396
	16	0.10887	0.160	0.0100
fine-grained με RW-lock	1	0.02526	1.00	1.00
	2	0.03417	0.739	0.370
	4	0.04561	0.554	0.139
	8	0.06367	0.397	0.0496
	16	0.12440	0.203	0.0127

Table 12: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 1000, συναλλαγές = 1000000, ποσοστό για ερωτήσεις υπολοίπου = 60%

Lock Type	Threads	Elapsed (s)	Speedup	Efficiency
coarse-grained με mutex	1	0.01728	1.00	1.00
	2	0.06217	0.278	0.139
	4	0.14611	0.118	0.0296
	8	0.19700	0.088	0.0110
	16	0.28260	0.061	0.0038
coarse-grained με RW-lock	1	0.00486	1.00	1.00
	2	0.04197	0.116	0.0579
	4	0.12575	0.039	0.0097
	8	0.39495	0.012	0.0015
	16	0.78199	0.006	0.0004
fine-grained με mutex	1	0.00481	1.00	1.00
	2	0.00869	0.554	0.2771
	4	0.03956	0.122	0.0304
	8	0.05570	0.086	0.0108
	16	0.09294	0.052	0.0032
fine-grained με RW-lock	1	0.01303	1.00	1.00
	2	0.03413	0.382	0.1909
	4	0.04471	0.291	0.0729
	8	0.06028	0.216	0.0270
	16	0.11593	0.112	0.0070

Table 13: Elapsed Time, Speedup και Efficiency μέγεθος πίνακα = 1000, συναλλαγές = 1000000, ποσοστό για ερωτήσεις υπολοίπου = 80%

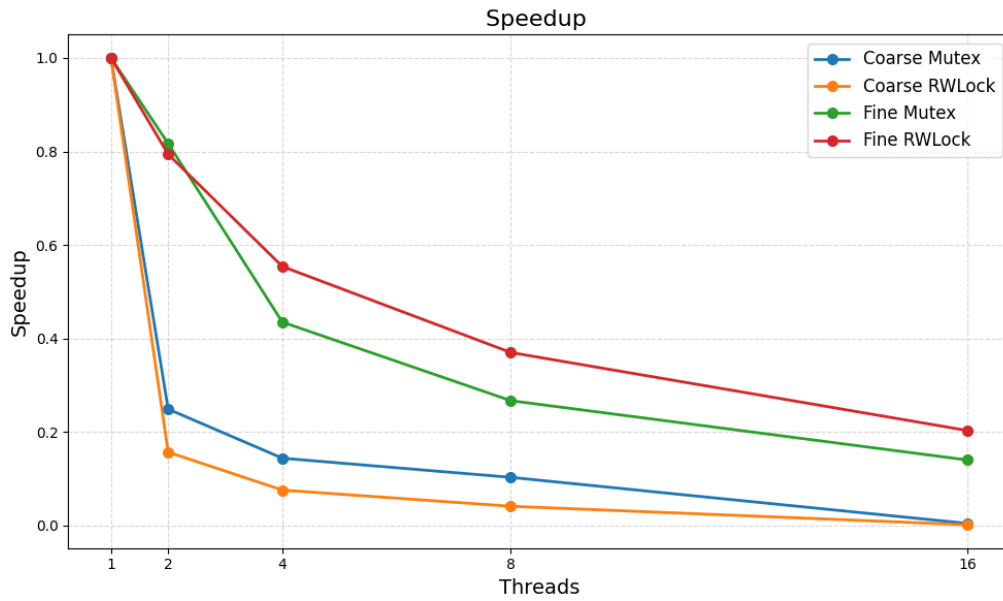


Figure 5: Speedup για μέγεθος πίνακα = 1000, συναλλαγές = 1000000, ποσοστό για ερωτήσεις υπολοίπου = 60%

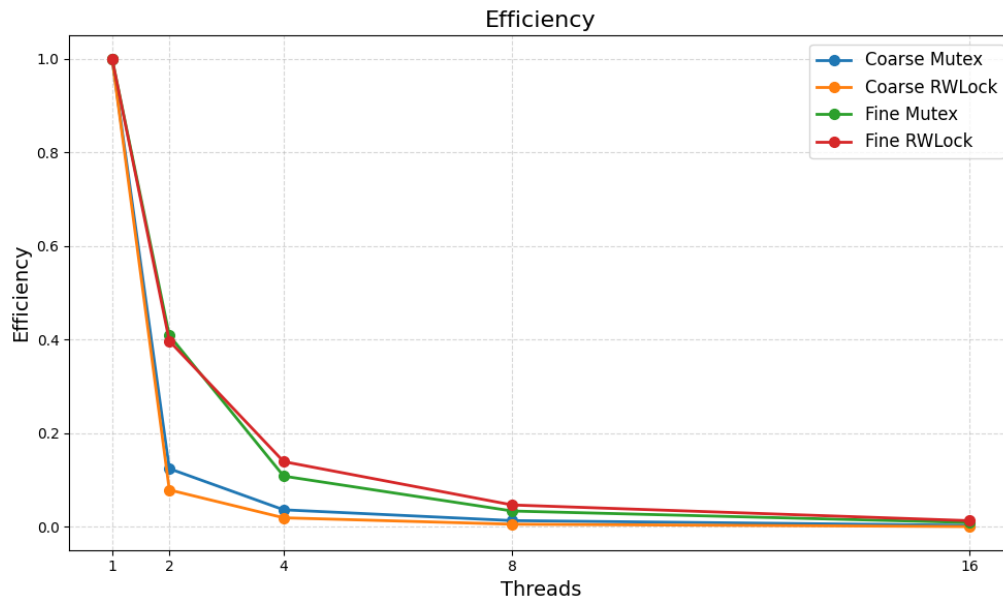


Figure 6: Efficiency για μέγεθος πίνακα = 1000, συναλλαγές = 1000000, ποσοστό για ερωτήσεις υπολοίπου = 60%

Από τα παραπάνω δεδομένα κάποιος μπορεί να συμπεράνει πως σε όλες τις περιπτώσεις η fine-grain είναι πιο γρήγορη και αποδοτική σε σχέση με τη coarse-grain σε όλες τις περιπτώσεις. Επιπλέον, από τα παραπάνω δεδομένα βλέπουμε πως όταν αυξάνεται το ποσοστό ερωτήσεων υπολοίπου δε βλέπουμε κάποια μεγάλη διαφορά μεταξύ των μηχανισμών κλειδώματος Mutex και rwlock. Αντίθετα από το αναμενόμενο παρατηρούμε πως ο μηχανισμός του rwlock είναι πιο λίγο πιο αργός σε σχέση με αυτόν του Mutex και αυτό συμβαίνει καθώς το critical section του κλειδώματος ανάγνωσης είναι πολύ μικρό και δε γίνεται κάποιο σημαντικό workload. Επομένως για να το δοκιμάσουμε και να συγκρίνουμε πραγματικά τους 2 μηχανισμούς θα καθυστερήσουμε το σημείο εντός του

critical section της ανάγνωσης (ερώτηση υπολοίπου) με την εντολή `usleep()`. Επιπλέον, θα διατηρήσουμε ένα υψηλό ποσοστό ερωτήσεων υπολοίπου της τάξεων των 90%. Σας παραθέτουμε τα αποτελέσματα:

Lock Type	Threads	Elapsed (s)
coarse-grained με mutex	8	118.93
coarse-grained με RW-lock	8	70.11
fine-grained με mutex	8	14.85
fine-grained με RW-lock	8	14.78

Table 14: Elapsed Time για μέγεθος πίνακα = 1000, συναλλαγές = 100000, ποσοστό για ερωτήσεις υπολοίπου = 90%

Έτσι συμπεραίνουμε πως οι υλοποιήσεις με κλειδώματα αμοιβαίου αποκλεισμού είναι αρκετά πιο αργές σε σχέση με τα κλειδώματα ανάγνωσης-εγγραφής. Πιο συγκεκριμένα, αυτό συμβαίνει στη coarse-grain καθώς στον mutex το κάθε νήμα περιμένει τον μοναδικό global mutex για να εισέλθει στο κρίσιμο σημείο ενώ με τα rwlock μπορούν πολλά νήματα να εισέλθουν στο κρίσιμο σημείο με read lock. Όσο αφορά την fine-grain υλοποίηση δε βλέπουμε κάποια σημαντική διαφορά καθώς η κάθε θέση έχει τον δικό της mutex ή rwlock με αποτέλεσμα σπάνια 2 νήματα να θέλουν να έχουν πρόσβαση στην ίδια θέση του πίνακα. Πάντως αν 2 νήματα ήθελαν να έχουν πρόσβαση στην ίδια θέση με ποσοστό ερωτήσεων υπολοίπου στο 90% η υλοποίηση με κλειδώματα ανάγνωσης-εγγραφής θα ήταν πιο γρήγορη.

Τέλος, παρατηρούμε πως για 8 και 16 νήματα η απόδοση σε όλες τις υλοποιήσεις (ανεξαρτήτου των εισόδων που θα τις δώσουμε) πέφτει αισθητά και αυτό συμβαίνει για 2 λόγους. Στα 8 νήματα σπάνια ο επεξεργαστής τρέχει ταυτόχρονα και τα 8 αφού μπορεί να χρησιμοποιεί και κάποιος άλλος το μηχάνημα. Στα 16 νήματα η απόδοση πέφτει ακόμη πιο πολύ αφού το μηχάνημα δε μπορεί να υποστηρίξει 16 νήματα να τρέχουν ταυτόχρονα(εμείς το υλοποιούμε και με 16 για πειραματικούς λόγους και πιο πλήρη δεδομένα).

6 Άσκηση 1.5

Πρόβλημα και Υλοποίηση

Σε αυτό το πρόβλημα καλούμαστε να δημιουργήσουμε n νήματα και να εκτελέσουμε με αυτά k επαναλήψεις. Μέσα σε κάθε υλοποίηση θα έχουμε ένα barrier και θα πειραματιστούμε με 3 διαφορετικούς μηχανισμούς φράγματος(`pthread_barrier`, φράγμα με τη χρήση κλειδώματος και μεταβλητής συνθήκης(`mutex` and `cond` και φράγμα που χρησιμοποιεί αναμονή σε εγρήγορση αλλά είναι επαναχρησιμοποιούμενο(`sense-reversal centralized barrier`)). Παρακάτω παραθέτουμε τα αποτελέσματα και τις παρατηρήσεις μου.

Threads	Υλοποίηση Barrier		
	Pthread (Time)	Mutex and Cond (Time)	Sense-Reversal (Time)
1	0.03287	0.000306	0.000112
2	0.03059	0.03420	0.000789
4	0.04305	0.2162	0.00618
8	0.08024	0.1575	0.0450
16	0.1670	0.3583	166.602

Table 15: Χρόνοι για διαφορετικές υλοποιήσεις barrier με 10000 loops ανά νήμα.

Threads	Υλοποίηση Barrier		
	Pthread (Time)	Mutex and Cond (Time)	Sense-Reversal (Time)
1	0.1162	0.001840	0.001855
2	0.2409	0.4013	0.07309
4	0.4198	0.6328	0.1434
8	0.7940	1.489	0.3599
16	1.660	3.579	– (δεν τερμάτισε σε λογικό χρονικό διάστημα)

Table 16: Χρόνοι για διαφορετικές υλοποιήσεις barrier με 100000 loops ανά νήμα.

Threads	Υλοποίηση Barrier		
	Pthread (Time)	Mutex and Cond (Time)	Sense-Reversal (Time)
1	0.8617	0.04907	0.04615
2	2.266	3.359	0.6019
4	3.992	6.099	1.400
8	8.000	14.61	2.957
16	16.59	36.30	– (δεν τερμάτισε σε λογικό χρονικό διάστημα)

Table 17: Χρόνοι για διαφορετικές υλοποιήσεις barrier με 1000000 loops ανά νήμα.

Με βάση τα αποτελέσματα μπορούμε να συμπεράνουμε πως ο βέλτιστος μηχανισμός για τη δημιουργία ενός φράγματος είναι το `sense-reversal centralized barrier`. Με την προϋπόθεση πως ο αριθμός των νημάτων που θα επιλέξουμε να μπορεί να εκτελεστεί ταυτόχρονα από τον επεξεργαστή (π.χ. στα πειράματά μας μέχρι 8 καθώς τόσα υποστηρίζει το μηχανήμα). Σε διαφορετική περίπτωση ο χρόνος ανεβαίνει ραγδαία καθώς όλα τα νήματα που κάνουν χρήση του `sense-reversal centralized barrier` κάνουν `busy waiting` με αποτέλεσμα να μη μπλοκάρει κανένα και για να τερματίσουν απλά να περιμένουν το δικό τους κβάντο χρόνου που θα τους ορίσει ο scheduler. Αντίθετα η υλοποίηση που παρέχει η βιβλιοθήκη της `pthread` ή ο συνδυασμός `mutex` και `condition variable` μπλοκάρει τα νήματα και αυτά "ζυπνάνε" όταν όλα τα υπόλοιπα έχουν φτάσει στο ίδιο σημείο(έχουν ξεπεράσει το φράγμα). Για αυτόν τον λόγο διακρίνουμε μια τεράστια αύξηση όταν χρησιμοποιούμε 16 νήματα. Επομένως, και τους 3 μηχανισμούς επηρεάζει το γεγονός ότι δημιουργούμε περισσότερα νήματα(16) από αυτά που μπορεί να εκτελέσει ταυτόχρονα ο επεξεργαστής(8) απλά στην περίπτωση του `sense-reversal centralized barrier` η αύξηση του χρόνου είναι πάρα δραματική λόγω του `busy waiting`.

Το sense-reversal centralized barrier είναι ένας αρκετά απλός μηχανισμός για τη δημιουργία ενός φράγματος. Αυτό που το κάνει επαναχρησιμοποιούμενο είναι το εξής:

Υπάρχει μια global μεταβλητή έστω ότι τη λένε flag. Κάθε νήμα παίρνει το flag και σε μια τοπική του μεταβλητή το αντιστρέφει και μένει σε ένα busy waiting μέχρι το flag και η τοπική του μεταβλητή να είναι ίσα. Το τελευταίο νήμα που θα φτάσει στο φράγμα θα αλλάξει τη τοπική του μεταβλητή και αφού είναι το τελευταίο θα αλλάξει και το flag. Έτσι όλα τα υπόλοιπα νήματα θα "ξεκολλήσουν" από το busy waiting και θα προχωρήσουν. Την επόμενη φορά που θα θελήσουν να χρησιμοποιήσουν το ίδιο φράγμα θα γίνει η ίδια διαδικασία. Επομένως, τη 1η φορά που θα χρησιμοποιήσουμε το flag με true, μετά το φράγμα θα γίνει false. Στη 2η φορά που θα πάμε στο φράγμα η τιμή του flag θα έχει παραμείνει true και μετά από αυτό θα αλλάξει σε false και αυτή η διαδικασία μπορεί να συνεχιστεί όσες φορές χρειάζεται. Έτσι το φράγμα καθίσταται επαναχρησιμοποιούμενο.

Κλείνοντας, αν αριθμός των threads είναι μικρότερος από τον αριθμό που μπορεί να τρέξει παράλληλα ο επεξεργαστής ο καλύτερος μηχανισμός για φράγμα είναι ο sense-reversal centralized barrier. Σε διαφορετική περίπτωση, ο καλύτερος μηχανισμός είναι αυτός της βιβλιοθήκης pthread.